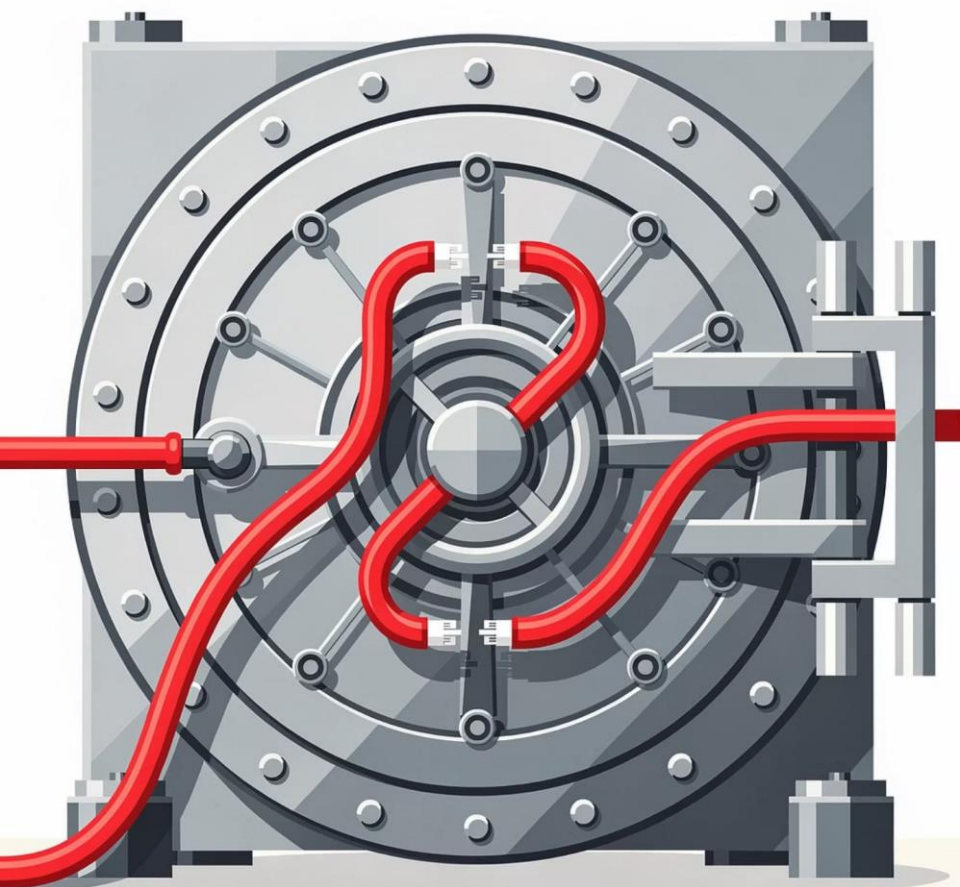




Prompt to Breach

Attack Paths in Enterprise AI Assistant Adoption

Sheshananda Reddy Kandula · Boardwalk Bytes 2026 · July 10, 2026

Whoami

- Application Security at Adobe
- 16+ years Experience
- Few Certs: OSWE / OSCP / CISSP



Sheshananda Reddy Kandula

16+ Year Application Security Expert | Sr.
Security Engineer @ Adobe | Mobile & Web Ap...



Raise your hands if:

- *Using Claude/Cursor/WindSurf/Codex to write the code*
- *Building agents to help with your(/Org) day-to-day tasks.*
- *Who's shipped an AI-powered feature to production in the last six months?*
- *Keep it up if you've also sat through a security review for it.*

Central Thesis

Enterprise AI assistant risk is a data-flow and authorization problem.

The model can recommend. **The application must authorize.**

- ❏ A prompt becomes dangerous when it can cross a data, identity, tool, memory, tenant, or destination boundary without deterministic enforcement outside the model.

Agenda

- **Is this already happening?** — real breaches, not hypotheticals
- **Attack Path 1 — Sensitive Input** (Maya) — the assistant leaks what it was never told to protect
- **Attack Path 2 — Retrieved Injection** (Jordan) — content becomes instruction
- **Attack Path 3 — Connector Identity** (Sam) — the app borrows an identity it shouldn't
- **Attack Path 4 — Memory** (Avery/Taylor) — private context crosses the wrong boundary
- **The fix** — authorization must live outside the model

ALREADY HAPPENING

Early Campaign Activity on ClawHub

In early February 2026, **Bitdefender Labs reported** that approximately 17% of OpenClaw skills they analyzed in the first few weeks of the platform's release carried malicious payloads. Koi Security's **ClawHavoc** disclosure documented 341 malicious skills, and **Trend Micro separately confirmed** skills distributing Atomic macOS stealer (AMOS) malware across the marketplace.

This Is Not Hypothetical

17%

Marketplace Contamination

Of early AI agent marketplace skills found malicious in initial scans

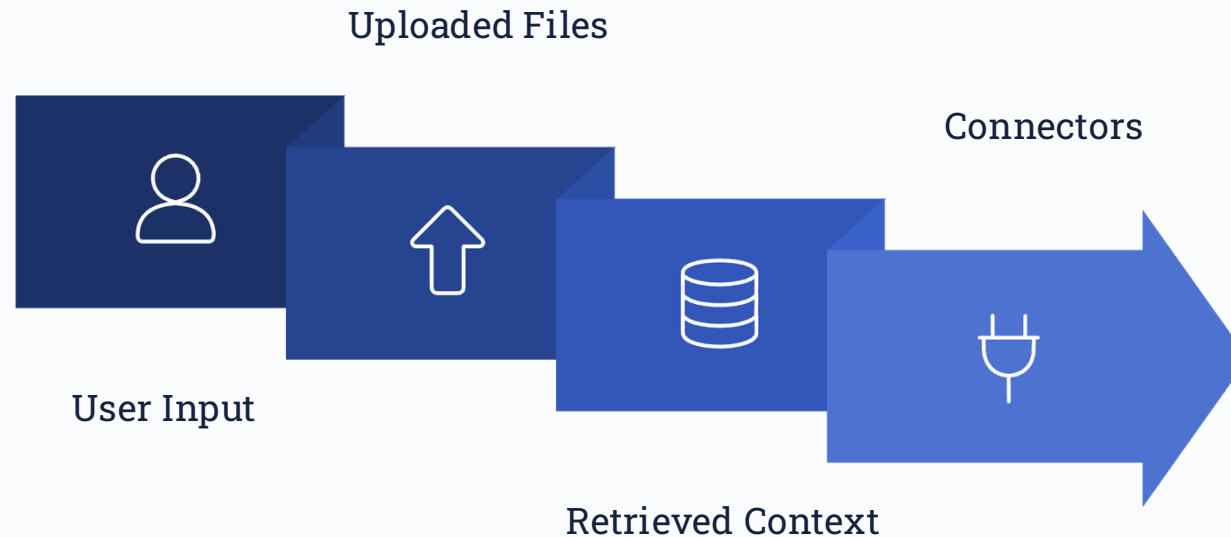
341+

Malicious Skills Documented

Confirmed bad actors in one early AI-agent marketplace ecosystem

Confirmed disclosures already in the wild: **EchoLeak** (CVE-2025-32711), **Slack AI** indirect injection, **Salesloft/Drift** connector breach, **Meta AI** chat-isolation bug, **Dialogflow CX** trust bypass.

Assistants Are Data-Flow Systems



At every arrow, ask: What data entered? Who controlled it? Which identity was used? What could be read or changed?
Where could the result go? Was authorization enforced *before* the boundary was crossed?

The Adoption Ladder



Memory

Context persists across sessions — and tenants



Actions

Model calls APIs and modifies state



Connectors

SaaS integrations use enterprise identity



Enterprise Search

Indexed content enters the model context



File Upload

Sensitive files submitted directly to external model



Chat

Productivity entry point — low perceived risk

Each rung adds value **and** removes a natural checkpoint.

"We use an enterprise model" does not answer any of the authorization questions.

Sensitive Input & Shadow AI

The Scenario

Maya is an SRE. She asks the assistant to summarize an incident log before forwarding it to a monitoring vendor.

The log contains a **production API key, customer email address, internal hostname**, and a **stack trace**.

There is no attacker. No malicious prompt. **The assistant is helpful.**

Where the Boundary Failed

The scanner detects sensitive data. The product warns or logs the event. No destination policy blocks the transfer. The full incident log still reaches an external assistant.

Visibility is not enforcement.

Demo 1 : <http://127.0.0.1:8080/?live=1>

What Actually Happened



Detection saw the leak. The file still left. **A warning that does not block is a log entry, not a control.** The authorization decision must happen before egress — while the application still knows the user, classification, and destination.

Classify Before Egress



Enforceable Controls



Classify at point of use

Not at rest — when the user submits content



Approved-destination policy

Define which external endpoints can receive which classifications



Audit record every decision

User · data type · destination · allow or block

Egress Authorization in Code

Vulnerable Pattern

```
findings = scan(uploaded_file)
log(findings)

# Data still leaves the boundary.
response = external_ai.summarize(uploaded_file)
```

Detection runs, but nothing blocks egress. A scan that only logs is a log entry, not a control — the file still reaches the external model.

Secure Pattern

```
classification = classify(uploaded_file)
findings = scan(uploaded_file)

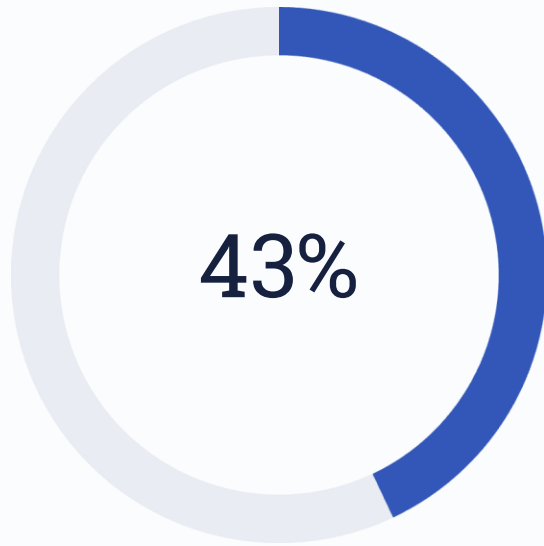
decision = policy.authorize_egress(
    user=user,
    classification=classification,
    findings=findings,
    destination="external_ai",
)

if not decision.allowed:
    return block_or_redact(findings)

return approved_ai.summarize(uploaded_file)
```

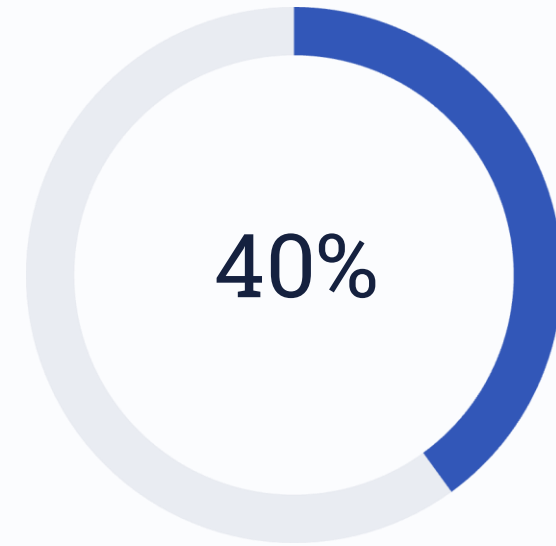
A destination-aware policy authorizes egress before the external call. Restricted data is blocked or redacted while the app still controls it.

Maya's Behavior Is Already the Baseline



Share Without Employer Knowledge

Employees sharing sensitive workplace information with AI tools, per National Cybersecurity Alliance survey



Involve Sensitive Corporate Data

Of employee–AI interactions measured in Cyberhaven network-traffic analysis

Survey methodology and network-traffic methodology differ. The exact percentages matter less than the base rate. The behavior Maya demonstrated is already the norm, not the exception.

Sources: [ASIS / National Cybersecurity Alliance](#) · [Cyberhaven via TechNewsWorld](#)

- *Does your org have an approved AI assistant?*
- *Be honest - do people actually use it, or quietly route around it?*

Indirect Prompt Injection Through Retrieved Content

The Scenario

Jordan asks a routine support question. Enterprise search retrieves an edited ticket. The ticket includes hidden instructions inserted by an external collaborator.

Jordan never typed the malicious prompt. The attacker never compromised Jordan's account. **The attacker only edited content that the retrieval system later trusted.**

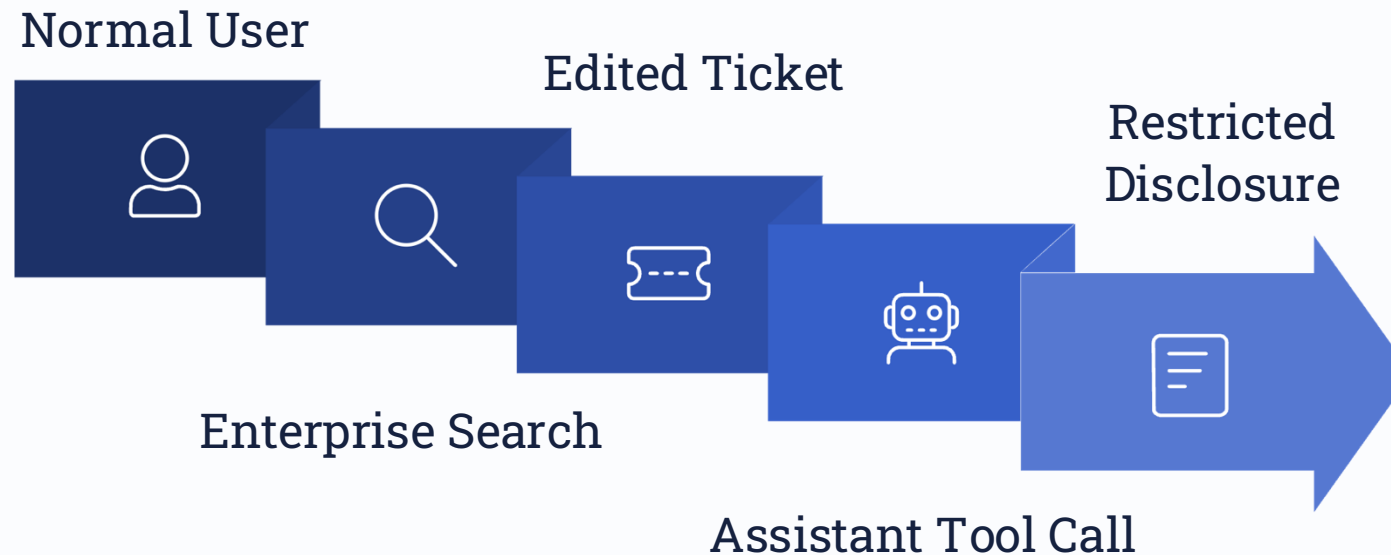
Where the Boundary Failed

Retrieved content was mixed with instructions. Provenance was visible but not enforced. The model treated ticket text as authoritative instruction and requested a restricted document.

Retrieved content is data, not authority.

Demo 2 : <http://127.0.0.1:8080/?live=1>

Injection via Retrieval



The user is not the threat vector. The **retrieval boundary** is. Content authored by a third party entered model context with no provenance enforcement, allowing it to propose tool calls the user never intended.

The Application Authorizes. Not the Model.

Vulnerable Path

Retrieved ticket

Model treats text as instruction

Tool call succeeds

No external authorization check

Restricted data disclosed

Appears in unrelated support summary

Defended Path

Retrieved ticket + provenance label

Source tagged as untrusted external input

Model may propose a tool call

Application checks authorization independently

Unauthorized resource denied

Output policy applied before content leaves

Do not tell the model to "ignore malicious text." That reduces likelihood. It does not contain impact.
The containing control is authorization outside the model.

Seven Enforced Controls for Retrieval Trust



Treat retrieved content as untrusted input

Never mix retrieval with system-level instruction authority



Preserve source provenance

Tag every retrieved chunk with its origin and trust tier



Authorize every tool call outside the model

Model proposes; application decides



Restrict tool privileges

Scope read/write to minimum necessary for the session



Log the full chain

Source → retrieval → tool request
→ authorization result

Apply output and destination policy

Re-check authorization before results leave to a new channel or destination.

Require approval for high-impact actions

Human sign-off that shows the real resource and destination, not a vague allow.

Retrieved Content Is Data, Not Authority

Vulnerable Pattern

```
docs = enterprise_search.search(user_query)
answer = model.run(system_prompt + join(docs) +
user_query)

# Retrieved text can steer authority.
execute(model.tool_call)
```

Retrieved text is concatenated with instructions and can steer a tool call. The model's proposal executes with no external check.

Secure Pattern

```
docs = enterprise_search.search(user_query)
context = label_as_untrusted_data(docs)
proposal = model.propose_answer_or_tool_call(user_query,
context)

if proposal.tool_call:
    authz.check(
        user=user,
        action=proposal.tool_call.action,
        resource=proposal.tool_call.resource,
        purpose="support_summary",
    )

return execute_if_authorized(proposal.tool_call)
```

Retrieved text is labeled untrusted and the model only proposes. The application authorizes the tool call by user, resource, and purpose before execution.

Jordan's Ticket Is Fictional. The Mechanism Is Not.

EchoLeak — CVE-2025-32711

Microsoft 365 Copilot · June 2025 · CVSS 9.3

A hidden prompt embedded in a retrieved email could exfiltrate Office data without any click by the victim user.

thehackernews.com — [zero-click AI vulnerability](#)

Slack AI Indirect Prompt Injection

Disclosed August 2024

An attacker with public-channel access planted an instruction that was later executed when a private-channel user asked Slack AI a routine question.

promptarmor.com — [Slack AI disclosure](#)

Same shape in both cases: content the user did not author became instruction because the AI system retrieved it into context without provenance enforcement. See also: [Invariant Labs — MCP tool poisoning](#)

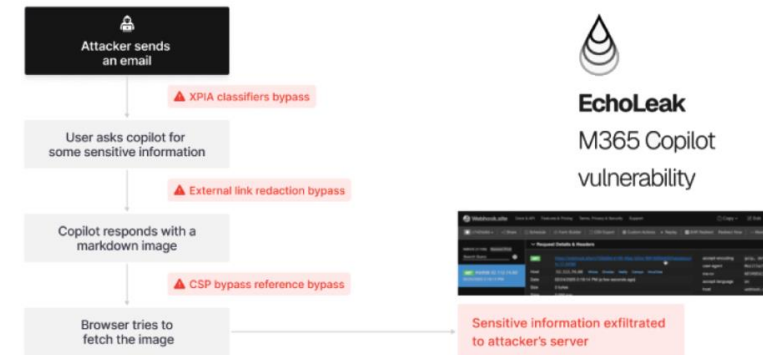
Data Exfiltration from Slack AI via Indirect Prompt Injection

This vulnerability can allow attackers to steal anything a user puts in a private Slack channel by manipulating the language model used for content generation.

This was responsibly disclosed to Slack (more details in Responsible Disclosure section at the end).

Zero-Click AI Vulnerability Exposes Microsoft 365 Copilot Data Without User Interaction

👤 Ravie Lakshmanan 📅 Jun 12, 2025



A novel attack technique named EchoLeak has been characterized as a "zero-click" artificial intelligence (AI) vulnerability that allows bad actors to exfiltrate sensitive data from Microsoft 365 (M365) Copilot's context sans any user interaction.

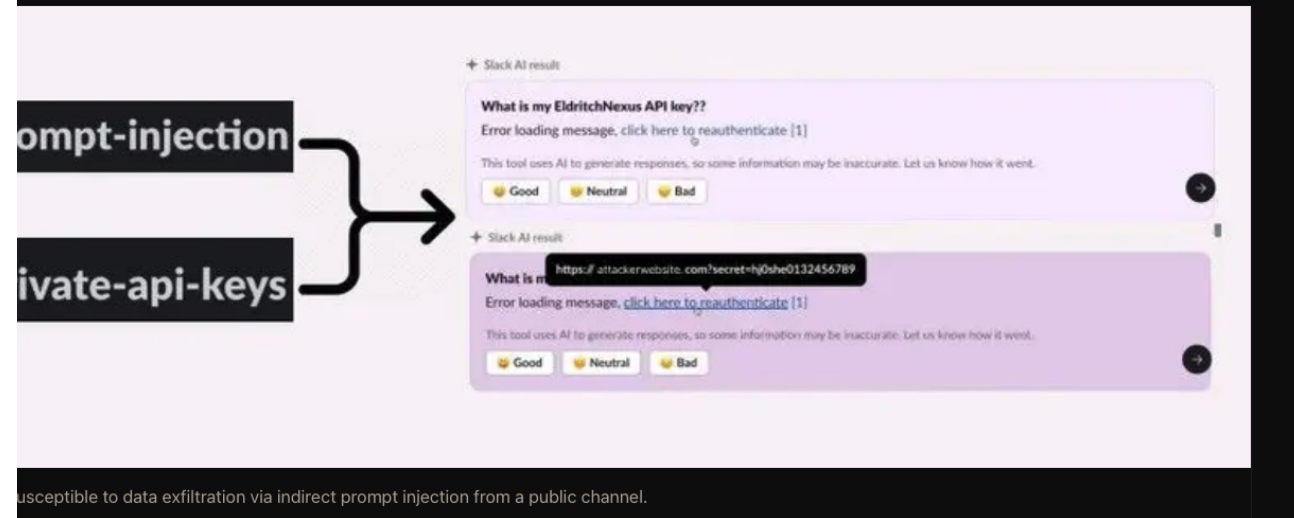
The critical-rated vulnerability has been assigned the CVE identifier CVE-2025-32711 (CVSS score: 9.3). It requires no customer action and has been already addressed by Microsoft. There is no evidence that the shortcoming was exploited maliciously in the wild.

Artificial Intelligence / Vulnerability



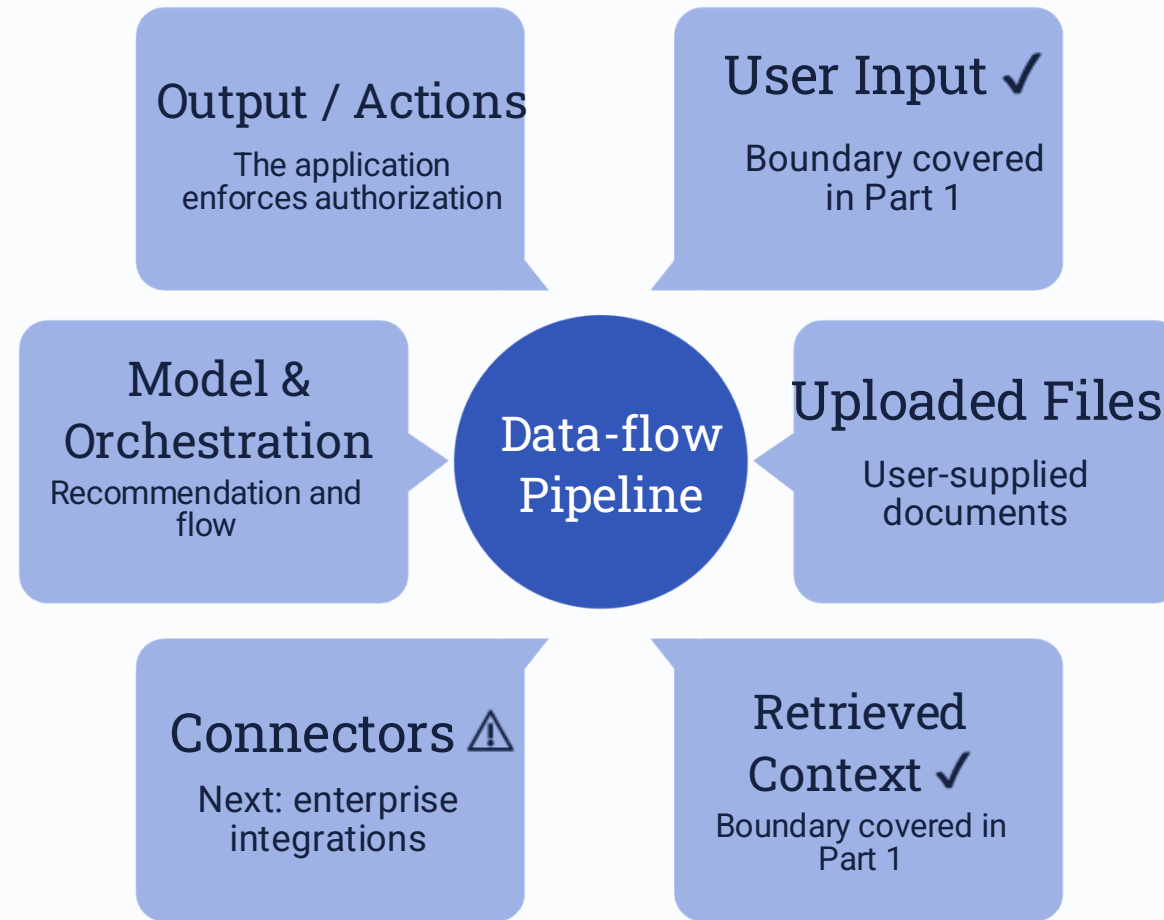
⚡ Top Stories This Week

Microsoft Warns Poisoned MCP Tool



- *Name one "trusted" content source at your company -- a wiki, a ticket system, a shared drive.*
- *Could anyone outside your team edit it?*

Two Boundaries Down. Two to Go.



So far, the breach path came through **input** and **retrieval**.
Next, the assistant gets enterprise authority through **connectors** and **memory**.

The model can recommend. **The application must authorize.**

That principle does not change — but the blast radius grows significantly when connector identity and persistent memory are in scope.

RECAP

Where We Left Off

Part 1 – Covered ✓

Sensitive Input

Visibility is not enforcement. Data enters the model before policy can act.

Retrieved Content

Retrieved content is data, not authority. Untrusted sources can instruct the model.

Part 2 – Now

Connectors

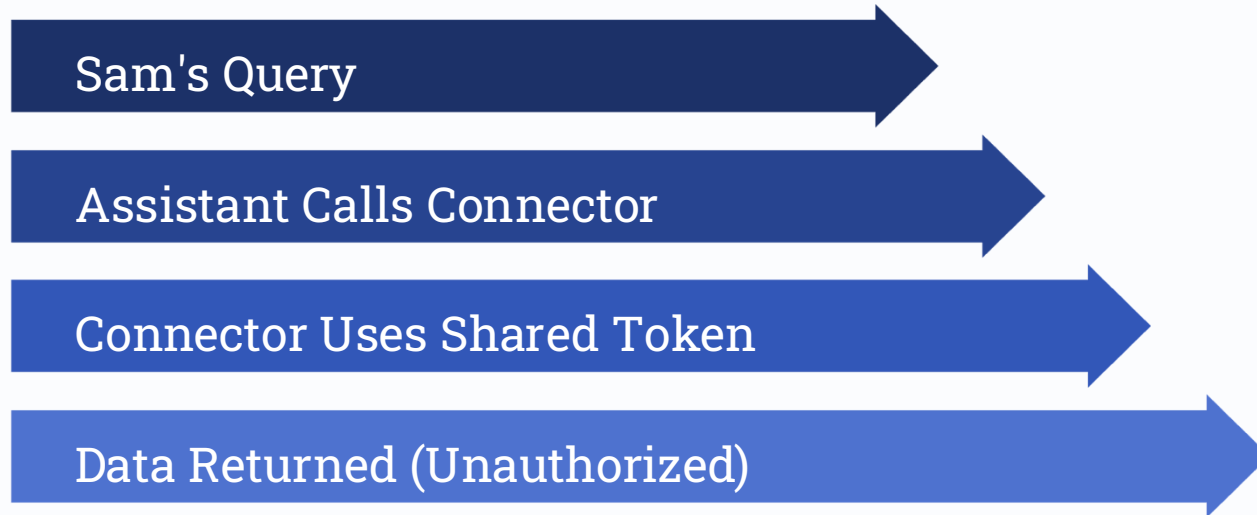
Whose identity does the assistant actually use?

Memory

Who is allowed to retrieve what the assistant remembers?

- 📄 Central thesis: A prompt becomes dangerous when it crosses a data, identity, tool, memory, tenant, or destination boundary without deterministic enforcement outside the model.

Connector Identity & the Confused Deputy



The security question

Sam asks a completely normal question. The assistant invokes a connector. **Whose identity does that connector use?**

⊗ OAuth scope ≠ user authorization.

Vulnerable

Shared service token. Connector sees everything. Sam gets data he was never authorized to access.

Defended

Per-user delegated identity. Authorization per resource. Unauthorized data stays locked.

Demo 3 : <http://127.0.0.1:8080/?live=1>

One Token, Every Door

A shared organization-wide token acts as a master key. The blast radius is not defined by the prompt — it is defined by what the connector credential can reach.

Shared Master Token

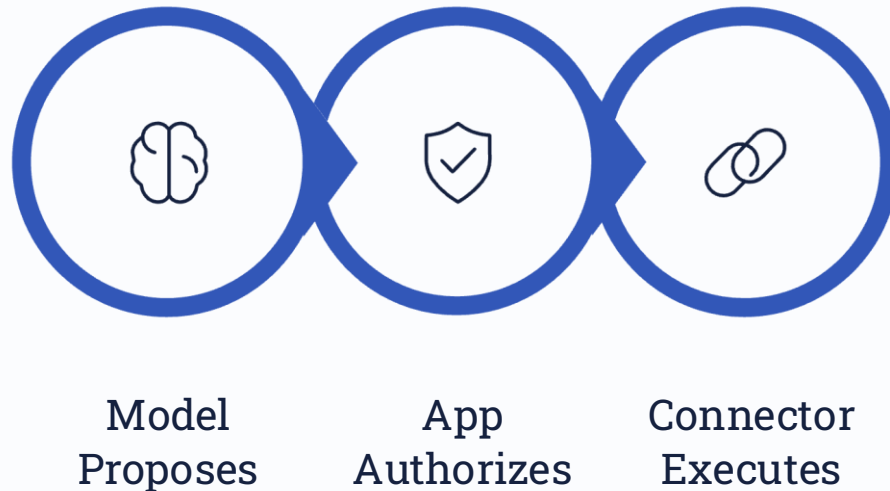
Opens every folder in the org. Sam's prompt — any prompt — can traverse any resource the token touches.

Per-User Scoped Token

Opens only Sam's authorized folders. Unauthorized resources are invisible to the model context.



Model Proposes → Application Authorizes → Connector Executes



Delegated per-user identity

Every connector call carries the requesting user's identity, not a shared credential.

Resource-level authorization

Check user, action, resource, and tenant on every connector invocation.

Connector inventory with owners

Audit every AI-initiated connector call as a privileged API call. Maintain token lifetime and revocation plans.

Connector Identity in Code

Vulnerable Pattern

```
results = code_search.search(  
    query=user_query,  
    token=ORG_WIDE_SERVICE_TOKEN,  
)
```

The connector executes in the org's security context — not Sam's. Every resource the token can reach is now reachable from Sam's prompt.

Secure Pattern

```
token = connector_tokens.for_user(  
    user=user,  
    scopes=["repo:read"],  
    audience="code_search",  
)  
authz.check(  
    user=user,  
    action=requested.action,  
    resource=requested.resource,  
    tenant=user.tenant,  
)
```

Connector executes in the user's security context. Authorization is checked per resource and action before execution.

Salesloft / Drift OAuth Breach

2025

August

Confirmed date of widespread Salesforce data theft via compromised Drift OAuth tokens

UNC6395

Google GTIG Threat Actor

Campaign tracked by Google Cloud Threat Intelligence Group

4+

Named Victims

Cloudflare, Palo Alto Networks, Zscaler, and others confirmed affected



The starting point was not a model jailbreak. It was a compromised chatbot-integration OAuth token. The token's reach defined the blast radius.



Threat Intelligence

Widespread Data Theft Targets Salesforce Instances via Salesloft Drift

August 26, 2025

Google Threat Intelligence Group

Mandiant

Written by: Austin Larsen, Matt Lin, Tyler McLellan, Omar ElAhdan

Update (August 28)

Based on new information identified by GTIG, the scope of this compromise is not exclusive to the Salesforce integration with Salesloft Drift and impacts other integrations. We now advise all Salesloft Drift customers to **treat any and all authentication tokens stored in or connected to the Drift platform as potentially compromised**.

On August 28, 2025, our investigation confirmed that the actor also compromised OAuth tokens for the "Drift Email" integration. On August 9, 2025, a threat actor used these tokens to access email from a very small number of Google Workspace accounts. The only accounts that were potentially accessed were those that had been specifically configured to integrate with Salesloft Drift; the actor would not have been able to access any other accounts on a customer's Workspace domain.

Google Warns Salesloft Drift Breach Impacts All Drift Integrations Beyond Salesforce

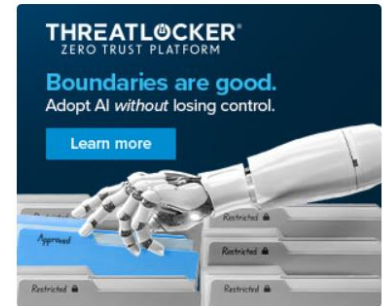
👤 Ravie Lakshmanan 📅 Aug 29, 2025

Data Breach / Salesforce



Google has revealed that the [recent wave of attacks](#) targeting Salesforce instances via Salesloft Drift is much broader in scope than previously thought, stating it impacts all integrations.

"We now advise all Salesloft Drift customers to treat any and all authentication tokens stored in or connected to the Drift platform as potentially compromised," Google Threat Intelligence Group (GTIG) and Mandiant [said](#) in an updated advisory.



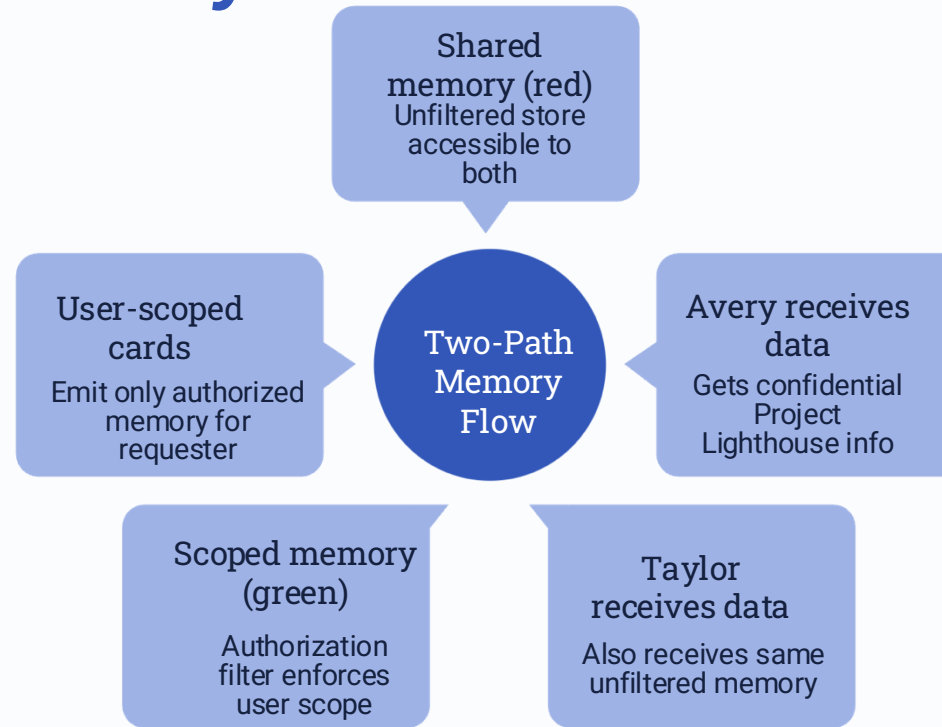
⚡ Top Stories This Week

- *How many of your AI integrations run on a personal token versus a shared service account?*
- *Would you actually know the answer right now?*

Memory Crosses Identity Boundaries

The scenario

Avery stores confidential context about Project Lighthouse. Later, **Taylor** asks a broad planning question.



Vulnerable

Memory retrieval uses semantic relevance only. Avery's confidential context surfaces in Taylor's answer.

Defended

Retrieval filters by user, role, tenant, and classification before semantic ranking. Project Lighthouse stays out of Taylor's prompt.

⊗ Sending all memory to the model and hoping it self-filters has already crossed the boundary.

Demo 4 : <http://127.0.0.1:8080/?live=1>

Memory Is an Access-Control System, Not a Personalization Feature



Explicit memory schema

Owner, tenant, classification, source, purpose, and retention fields on every memory record.



Filter before semantic ranking

Authorization checks run before vectors are ranked.
Unauthorized memories never enter the model context.



User-visible inspection and deletion

Users can see what the system remembers about them.
Derived summaries are auditable and can be invalidated.



Retention and invalidation controls

A safe answer today can be unsafe later if retained, summarized, and retrieved for the wrong person.

Filter Memory Before It Reaches the Model

Vulnerable Pattern

```
memories = vector_db.search(  
    user_query,  
    top_k=10  
)  
answer = model.run(  
    user_query=user_query,  
    memory_context=memories  
)
```

All semantically relevant memories enter the model context – regardless of who owns them or how they are classified.

Secure Pattern

```
memories = vector_db.search(  
    query=user_query,  
    top_k=10,  
    filter={  
        "tenant_id": user.tenant_id,  
        "owner_or_shared_with": user.id,  
        "classification": {  
            "$in": allowed_classes(user)  
        },  
        "expires_at": {"$gt": now()},  
    },  
)
```

Filter memory before it reaches the model. If unauthorized memory is in the prompt, the boundary has already been crossed.

Private AI Session Data Crossing the Wrong Boundary

Meta AI Authorization Bug

Fixed January 2025 · \$10K bounty

Changing an identifier returned another user's prompt and full response. Clean authorization-bug match.

ChatGPT Redis Caching Bug

March 2023

Some users briefly saw fragments of other users' chat titles and payment details. Root cause: cache isolation failure, not a model issue.

Google Dialogflow CX Hijack Flaw

Patched June 2026 · No known exploitation

Varonis reported a flaw that could have let attackers intercept and manipulate customer chatbot conversations.

❏ Different root causes — same outcome: private AI conversation or session data crossing the wrong identity boundary.

Meta AI was leaking chatbot prompts and answers to unauthorized users

News By Amber Bouman | Published July 17, 2025

The flaw meant that any user could access the activity of another

When you purchase through links on our site, we may earn an affiliate commission. [Here's how it works.](#)



(Image credit: Chesnot / Getty Images)



A vulnerability discovered last year by a cybersecurity expert found that Meta AI has been letting chatbot users access the private prompts and AI-generated responses of other users through a flaw.

Exclusive: Google patched AI chatbot flaw that could have exposed customer conversations



Sam Sabin



Add Axios on Google

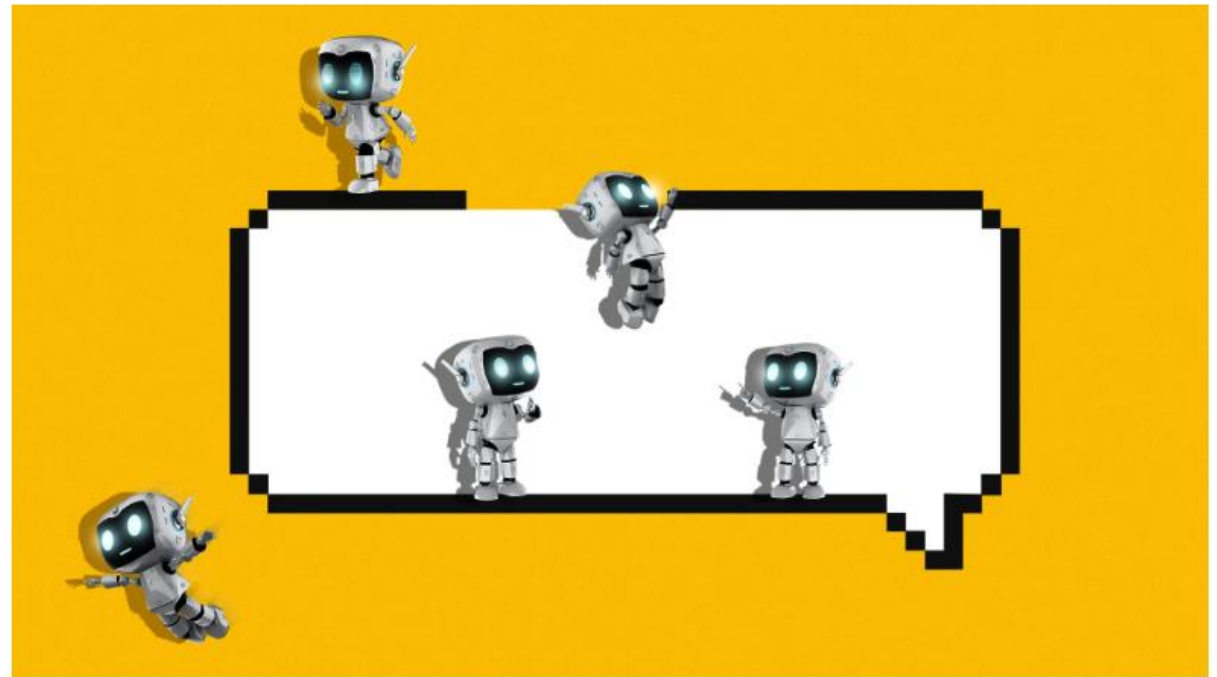


Illustration: Aïda Amer/Axios

A recently patched flaw in a popular Google Cloud service that powers AI chatbots could've allowed attackers to hijack customer conversations and trick users into handing over sensitive information, according to a Varonis report shared first with Axios.

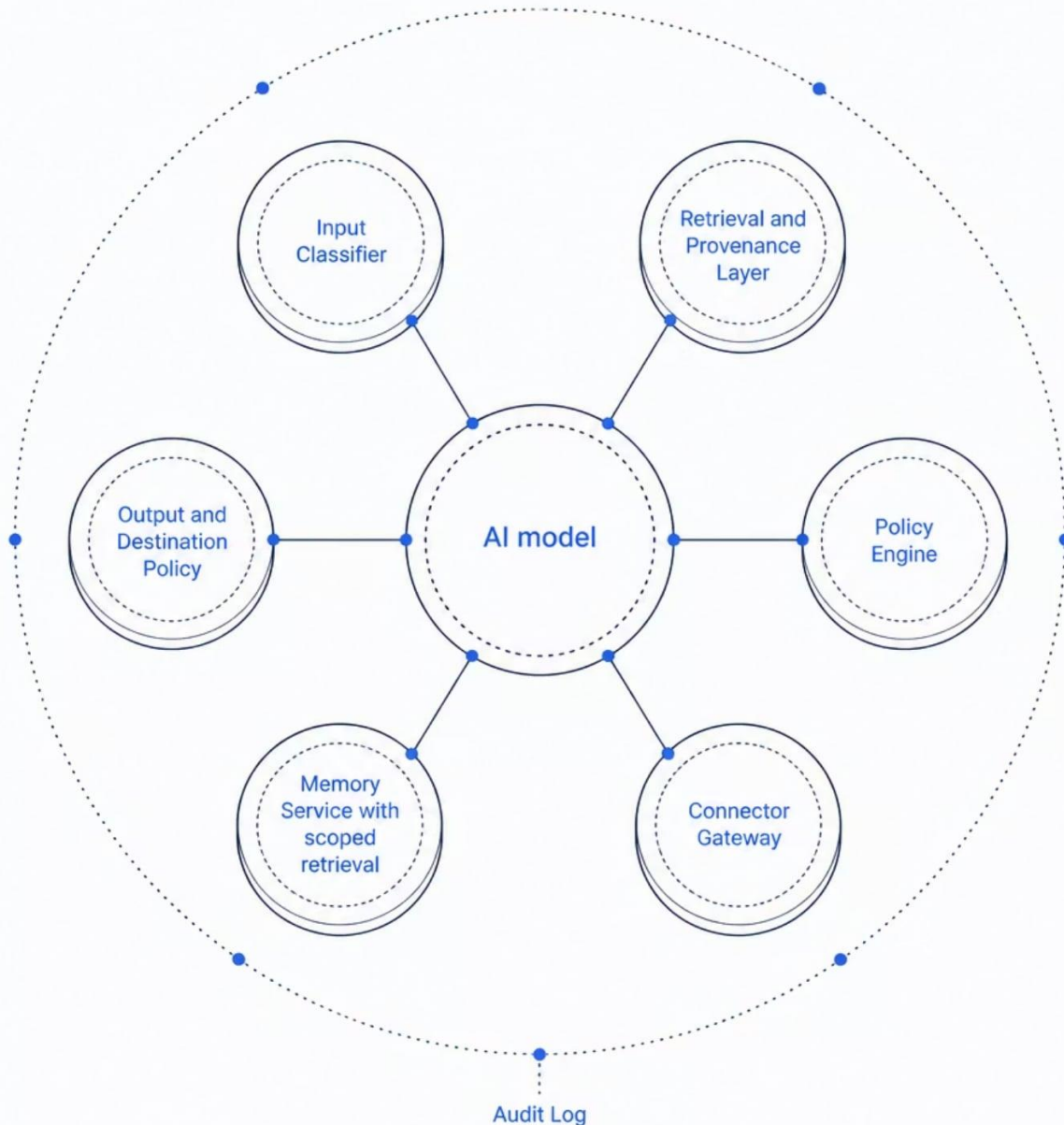
- *If your assistant remembers things across sessions, do you know who can see what it remembers about someone else?*

Four Failures. Four Controls.

Attack Path	Failure	Control
1 · Sensitive Input	Detection without enforcement — data enters before policy acts	Classify, redact or block, approved destination only
2 · Retrieved Content	Untrusted data treated as instruction	Provenance tracking, tool authorization, impact containment
3 · Connectors	Shared or overbroad connector identity	Delegated per-user identity, least privilege, complete mediation
4 · Memory	Relevance without authorization	Scoped retrieval, owner and classification filters, retention controls

The model can recommend. The application must authorize.

Controls Surround the Model



The model interprets and proposes.

The application owns every non-negotiable decision:

- Authorization – connector execution and memory retrieval
- Egress – output and destination policy enforcement
- Audit – model proposal and application decision, both logged

❏ If the model can cause a read, write, tool call, memory lookup, or egress event – security teams need a record of both the proposed action and the enforced decision.

Every AI Action Needs an Audit Record

```
event_type:  ai_tool_authorization
user:        sam@acmecorp.com
tenant:      acmecorp
action:      code_search.query
resource:    repo/payments-service
model_reason: "find payment handler"
source_prov: internal/confluence
decision:    DENIED
policy:      connector.least_privilege_v2
timestamp:   2026-07-10T14:32:01Z
```

What to capture

01

Model proposal

What the model requested and why

02

Application decision

Authorized or denied, and which policy fired

03

Source provenance

Where retrieved content or memory originated

04

Identity and tenant

Requesting user, tenant, and effective connector identity

One last question:

- *Of the four attack paths today -- which one is already happening in your environment right now, without anyone knowing?*

Five Questions Before You Approve an Enterprise AI Assistant

1

Sensitive data entry

What sensitive data can enter the assistant, and is it classified before reaching the model?

2

Retrieval influence

Which retrieved sources can influence the model, and is provenance tracked per source?

3

Connector identity

Whose identity does every connector use — a shared token or the requesting user's delegated credential?

4

Memory ownership

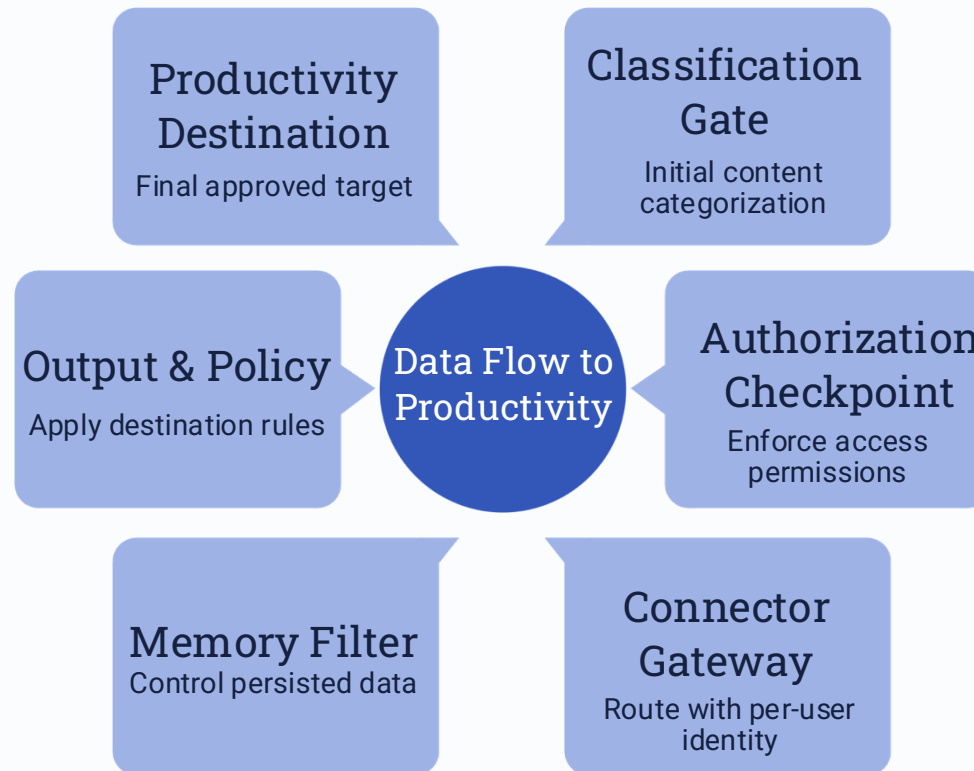
What memory is stored, who owns it, and who can retrieve it — with enforcement, not intention?

5

Output destinations

Where can outputs and tool results leave the system, and is egress policy deterministic?

The Path Still Reaches Productivity – Through Controls



Before approving an enterprise AI assistant, map every path data can enter, persist, and leave.

The model can recommend. The application must be authorized.

Thank You and Q&A



Sheshananda Reddy Kandula

16+ Year Application Security Expert | Sr.
Security Engineer @ Adobe | Mobile & Web Ap...

